

STACKS & QUEUES

Task 01

Your friend has a strange habit of borrowing books from everyone but returning absolutely none of them. One day, you decide to investigate his latest collection of borrowed books, represented by the string "BORROWROB".

Implement a Stack using Arrays and insert each character of the string "BORROWROB" into the stack using the push() function. Then, use the pop() function to retrieve the characters and determine whether the given string is a palindrome or not.

Clearly display the stack operations and final result.

Hint: A palindrome reads the same forwards and backwards, unlike your friend's borrowing habits.

Note: You must implement the stack using arrays and use the push and pop functions to accomplish the task.

Task 02

Suppose you are developing a task management system for a busy professional whose to do list grows faster than their motivation to complete it. To make matters worse, they insist that the most recently added task must be dealt with first because apparently older tasks have already been ignored for too long.

You are required to implement a Stack data structure to manage these tasks.

Write a complete code snippet that performs the following operations:

1. Add the tasks "Reply to Emails", "Attend Meeting", "Complete Report", and "Submit Assignment" to the stack using the push() function.
2. Before removing any task, check whether the stack is empty.
3. Remove tasks from the stack using the pop() function and display them in the order they are removed.
4. After removing two tasks, check again whether the stack is empty.
5. Finally, remove all remaining tasks and verify the stack's empty condition.

Your solution must clearly demonstrate the LIFO principle and handle all empty stack conditions appropriately.

Note: Implement the Stack using a suitable data structure. The professional may have no intention of completing the tasks, but your program must still manage them correctly.

Task 03

Suppose you are developing a web browser history manager for a user who opens multiple websites while pretending to work, only to realize later that they have accidentally visited five different social media platforms instead of completing their assignment.

To manage this browsing history, the program uses both a **Linked List** and a **Stack**.

Whenever the user visits a new website:

- The website URL is inserted at the front of the Linked List to represent the most recently visited page.
- The same URL is pushed onto the Stack to support the browser's Back button functionality.

The user visits the following websites in this exact order:

Google → Facebook → Twitter → LinkedIn → Instagram

Since the user has finally realized that the assignment is due tomorrow, they decide to click the Back button twice.

Task:

Write a function that implements the Back button behavior using both the Linked List and Stack.

Your function must:

1. Pop two elements from the Stack.
2. Remove the first two nodes from the Linked List.
3. Display the website the user is currently viewing after pressing Back twice.
4. Ensure that both data structures remain synchronized after the operation.

Additional Challenge:

What would happen if the user presses the Back button three more times after returning to Facebook? Your implementation should handle this situation appropriately without causing a crash or accessing an empty Stack.

Hint: The user may have forgotten where they were browsing, but your data structures must not.

Task 04

You have been hired to develop an order management system for a restaurant that has suddenly become extremely popular. Orders are arriving faster than the kitchen staff can process them, and customers are becoming increasingly impatient.

The restaurant wants orders to be processed **in the exact order in which they were received**. Each order contains a customer name, order number, and the items ordered along with their respective quantities.

As the developer, answer the following:

1. Identify the most appropriate **data structure** for managing the restaurant orders and justify your choice.
2. Describe the operations required to **add a new order** and **remove an order** while maintaining the required processing order.
3. Explain how you would store the **quantity of each item** within an order.
4. Explain how you would prevent **overflow** when the data structure becomes full and **underflow** when there are no orders available for processing.
5. Describe how you would **display all pending orders** in the same order in which they were received, so that the kitchen staff knows exactly which customer should be served next.
6. Consider the situation where the restaurant receives a new order when the data structure is already full. Explain how your implementation should handle this situation without crashing or quietly deleting someone's biryani order.

Note: Your answer should clearly demonstrate your understanding of the **FIFO principle** and the relevant insertion, deletion, overflow, underflow, and traversal operations.

Task 05

You are working as a cashier at a supermarket where customers have somehow decided that standing in line is an excellent way to spend their evening. To keep things organized, you decide to implement a **Queue using Arrays** so that customers are served in the same order in which they arrive.

The Customer IDs are:

13, 7, 4, 1, 6, 8, 10

Task:

Implement a **Queue using Arrays** and perform the following operations:

1. Insert all Customer IDs into the Queue using the enqueue() operation.
2. Remove the customers from the Queue using the dequeue() operation.
3. Display each Customer ID as it is served.
4. Clearly demonstrate that customers are served according to the **FIFO principle**.
5. Handle the appropriate **overflow and underflow conditions** in your implementation.

Note: You must use an **Array based Queue** and implement the enqueue() and dequeue() operations. No customer should be allowed to mysteriously skip the queue simply because they are in a hurry.

Task 06

Suppose you are developing a messaging application where users are sending messages faster than the application can process them. Unfortunately, the application has one major limitation:

it can process only one message at a time. Apparently, multitasking was not included in the budget.

Each message may take a **different amount of time** to send or receive. As messages arrive, they are placed into a queue. Once the current message has been completely processed, the application selects the next message according to the queue's scheduling strategy.

Task:

1. Identify the **most suitable type of Queue** for this scenario.
2. Justify your choice based on the fact that messages require **variable processing times**.
3. Explain how messages should be **inserted and removed** from the queue.
4. Explain how your selected queue ensures that a newly arrived message does not unnecessarily disrupt a message that is already being processed.
5. Briefly describe what could happen if a simple **FIFO Queue** were used instead.

Note: Your answer should focus on the queue type and its processing strategy. The application may be slow, but at least the messages should not be processed in a completely chaotic order.

Task 07

You are developing a **library management system** where students line up to borrow or return books. The library follows **FIFO**, because apparently even the student saying "*Sir bas 30 seconds lagenge*" cannot be allowed to skip the queue.

Implement a **Linear Queue using an Array in C++** with a maximum capacity of 5.

The program should support:

1. enqueue() to add a Patron ID.
2. dequeue() to remove the patron from the front.
3. display() to show the current queue.
4. Proper handling of **overflow and underflow**.

Initially, enqueue the following Patron IDs:

101, 205, 317, 422, 509

Then perform the following operations:

Dequeue twice, enqueue 614, enqueue 728, dequeue until the queue becomes empty, and attempt one additional dequeue.

Display the queue after the major operations.

Additional Challenge: After the first two patrons are removed, empty spaces exist at the beginning of the array. Explain why a **Linear Queue** may still report overflow when trying to insert a new patron.

Note: Empty array positions do not magically become available again in a Linear Queue. Unlike the students, the queue does not simply move backwards when it realizes it made a mistake.